



Programa Certified AI-LLM Solution Architect

Fundamentos de Arquitectura LLM

Sesión 4 · Ejecución de llamadas API con buenas prácticas

www.bsginstitute.com

Andrés Felipe Rojas Parra

- Ingeniero electrónico con mas de 25 años de experiencia
- PGP Artificial Intelligence & Machine Learning at Texas University – Austin, TX
- Master en Inteligencia Artificial y Big Data – IEBSchool
- Estudiante de PGP en Cloud Computing at Texas University - Austin, TX
- Texas Executive Education Program – McCombs School of Business – Decision Science and AI Immersion Program
- Especialista en MLOps de la Universidad de Duke
- Estudiante de Deep Learning for Healthcare Specialization
- Director de Inteligencia Artificial e Innovacion Universidad EAN/Colombia



anrojas@universidadean.edu.co / arojaspa@outlook.com / arojaspa76



@arojaspa



<https://www.linkedin.com/in/arojaspa/>



+573005906373



Ejecución de Llamadas API con Buenas Prácticas

Parámetros, seguridad OWASP, prompting con respaldo en investigación y agentes en la nube

Ubicación en el Curso

La Sesión 3 dejó lista la infraestructura de inferencia: contenedores, Kubernetes y proveedores cloud desplegados y escalando.

Esta sesión asume esa infraestructura disponible y se enfoca en CÓMO llamarla correctamente: parámetros de generación, seguridad básica, técnicas de prompting con respaldo en investigación, control de alucinación y agentes.

PROGRESO DEL CURSO · 10 SESIONES

1

2

3

4

5

6

7

8

9

10

Agenda — Sesión 4

Tres bloques de trabajo · 3 horas de alto impacto

01

70 min

Fundamentos y Seguridad

- ▶ Anatomía de la llamada: system, user, parámetros, respuesta
- ▶ Matemática del muestreo: temperature y top_p
- ▶ Marco de decisión de parámetros por tipo de tarea
- ▶ Seguridad básica bajo OWASP Top 10 para LLMs

02

70 min

Prompting y Anti-Alucinación

- ▶ Few-shot, Chain-of-Thought y Zero-shot-CoT (con respaldo en investigación)
- ▶ Role Prompting y Structured Output como confiabilidad
- ▶ Por qué ocurre la alucinación + 5 técnicas de control
- ▶ Casos reales: Mata v. Avianca y Moffatt v. Air Canada

03

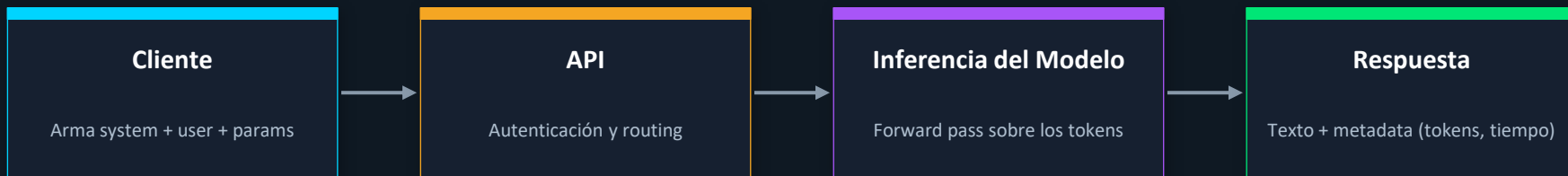
40 min

Demo Guiada y Agentes en GCP

- ▶ Arquitectura de la demo guiada (FastAPI + React 18 + Vite 6)
- ▶ Patrón ReAct: el loop de razonar-actuar-observar
- ▶ 3 implementaciones del mismo patrón en Google Cloud
- ▶ Síntesis: un marco integral para el arquitecto de LLM

La Anatomía de una Llamada API

Tema 1 · El flujo request → response y sus componentes



Piezas del Request

system define rol, tono y límites del asistente

user la instrucción o pregunta concreta

parameters temperature, max_tokens, top_p...

headers Authorization — la API key, nunca en el body

Consideraciones de Producción

Statelessness

cada llamada es independiente; el modelo no retiene memoria implícita entre requests.

Idempotencia

reintentos seguros ante fallos de red exigen diseño explícito (ej. un request-id).

→ ver ejemplos/01_estructura_llamada_api.py

Parámetros de Generación: la Matemática del Muestreo

Tema 1 · Qué le hacen `temperature` y `top_p` a la distribución de probabilidad



Temperature — escalado de logits

El modelo produce un logit por token candidato antes del softmax. `temperature` divide esos logits: un valor alto aplana la distribución (probabilidades más parejas → más aleatoriedad); un valor bajo la concentra en los tokens más probables (más determinismo).

`temperature=0.2`

→ salida casi determinista: apta para extracción de datos



Top-p — truncamiento de la cola

En vez de fijar cuántos tokens considerar, `top_p` ordena los candidatos por probabilidad y conserva el conjunto más pequeño cuya probabilidad acumulada supera `p` — el 'núcleo' — descartando la cola larga e improbable (nucleus sampling).

`top_p=0.9`

→ rara vez se ajusta junto con `temperature`



Holtzman et al., 2020 — "The Curious Case of Neural Text Degeneration", ICLR 2020, arXiv:1904.09751 — fundamento formal del muestreo por núcleo (top-p).



Buena práctica: fijar `temperature` y `top_p` explícitamente — nunca depender de los defaults del proveedor.

Marco de Decisión de Parámetros

Tema 1 · Qué configurar según el tipo de tarea — y por qué

Tipo de tarea	temperature	top_p	max_tokens	Justificación
Extracción / clasificación	0.0 – 0.2	0.1 – 0.3	128 – 256	Distribución casi determinista: hay una respuesta correcta, no se busca creatividad.
Chat / soporte al cliente	0.4 – 0.6	0.8 – 0.9	400 – 600	Balance: coherencia conversacional con variación natural en la redacción.
Generación creativa	0.8 – 1.0	0.95 – 1.0	700 – 1200	Distribución deliberadamente aplanada: se busca diversidad léxica y sorpresa.

La columna de justificación conecta directo con el mecanismo de la diapositiva anterior: *temperature* y *top_p* controlan qué tan concentrada está la distribución de la que se muestrea.

 Regla práctica: fijar *temperature* y *top_p* explícitamente para cada tipo de tarea — nunca depender de los defaults del proveedor.

Seguridad Básica bajo el Marco OWASP Top 10 para LLMs (2025)

Tema 1 · El estándar de la industria — y dónde vive en nuestra propia arquitectura

LLM01 Prompt Injection	LLM02 Sensitive Information Disclosure	LLM03 Supply Chain	LLM04 Data and Model Poisoning	LLM05 Improper Output Handling
LLM06 Excessive Agency	LLM07 System Prompt Leakage	LLM08 Vector and Embedding Weaknesses	LLM09 Misinformation	LLM10 Unbounded Consumption

Mapeo explícito a la arquitectura de esta sesión

- LLM02** → API keys vía `os.getenv()` / gestor de secretos — nunca hardcoded en el código.
- LLM10** → Rate limiting (`slowapi`) + techos explícitos de `max_tokens`.
- LLM01** → Sanitización del input del usuario antes de que llegue al prompt.

Técnicas de Prompting I — con Respaldo en Investigación

Tema 2 · Tres técnicas, tres papers, tres mecanismos distintos

01 Few-Shot Prompting

Incluir 2–3 ejemplos entrada→salida en el prompt le basta al modelo para inferir el patrón, sin actualizar ningún peso — aprendizaje en el contexto, no en el entrenamiento.

"Ejemplo 1: ... → ... Ejemplo 2: ... → ... Ahora: ..."

Brown et al., 2020 — "Language Models are Few-Shot Learners" (GPT-3), NeurIPS 2020, arXiv:2005.14165

02 Chain-of-Thought (CoT)

Pedir razonamiento explícito paso a paso mejora tareas aritméticas, simbólicas y de razonamiento multi-paso. La ganancia NO es uniforme: en tareas simples de una sola operación, aporta poco o nada.

"Explica tu razonamiento paso a paso antes de dar la respuesta final."

Wei et al., 2022 — "Chain-of-Thought Prompting Elicits Reasoning in LLMs", NeurIPS 2022, arXiv:2201.11903

03 Zero-Shot-CoT

Agregar solo la frase 'pensemos paso a paso' recupera gran parte de la ganancia de CoT sin necesitar ejemplos previos — la técnica de razonamiento casi gratuita.

"Pensemos paso a paso."

Kojima et al., 2022 — "Large Language Models are Zero-Shot Reasoners", arXiv:2205.11916

Técnicas de Prompting II — Role Prompting y Structured Output

Tema 2 · Una ajusta el tono; la otra es una técnica de confiabilidad

Role Prompting

Asignar un rol o persona ("actúa como...") condiciona la distribución de salida hacia el vocabulario, tono y prioridades de ese rol. Es una palanca de seguimiento de instrucciones, no un truco cosmético.

```
"Actúa como auditor financiero senior."
```

Structured Output

Exigir un esquema de salida (JSON, función) no es estética de formato: es una técnica de CONFIABILIDAD. Vuelve la respuesta parseable de forma determinista por el sistema — exactamente lo que un agente necesita para invocar tools sin fallos de parsing.

```
response_format={"type": "json_object"}
```

💡 Los agentes (más adelante en esta sesión) dependen de structured output para invocar herramientas de forma confiable — este es el hilo que conecta con la sección de agentes.

Por Qué Ocurre la Alucinación

Tema 2 · No es un bug aislado — es una consecuencia del objetivo de entrenamiento

El Mecanismo

El entrenamiento optimiza la verosimilitud del siguiente token: maximizar $P(\text{token siguiente} \mid \text{contexto})$. No existe una restricción explícita de verdad factual en ese objetivo. El modelo no 'sabe' que miente: genera la secuencia más probable, sea o no correcta.

La Evidencia Empírica

TruthfulQA muestra que los modelos pueden reproducir con total confianza concepciones erróneas comunes: la alucinación no es un riesgo anecdótico, es un fenómeno medido y estudiado en un benchmark diseñado para eso.

Lin, Hilton & Evans, 2022 — "TruthfulQA: Measuring How Models Mimic Human Falsehoods", ACL 2022, arXiv:2109.07958



Esto es lo que justifica tomar en serio las 5 técnicas de control de la siguiente diapositiva — no son un 'nice to have'.

Las 5 Técnicas de Control, con su Mecanismo

Tema 2 · No basta con nombrarlas — hay que entender por qué funcionan

Grounding / RAG

Aumentar el prompt con contexto recuperado y verificado antes de generar, en vez de confiar solo en los pesos del modelo.

Lewis et al., 2020 — RAG, NeurIPS 2020, arXiv:2005.11401

Self-Consistency

Muestrear varias cadenas de razonamiento independientes y quedarse con la respuesta mayoritaria entre ellas.

Wang et al., 2022 — arXiv:2203.11171

Exigir citas o fuentes

Forzar al modelo a señalar de dónde viene cada afirmación hace la respuesta verificable por un humano, no solo plausible.

Permitir "no lo sé"

Remover el incentivo implícito a siempre completar una respuesta plausible cuando falta contexto suficiente.

Temperature baja en tareas factuales

Distribución más concentrada → menos variación → menor probabilidad de deriva hacia una alternativa inventada.

Mata v. Avianca, Inc.

678 F. Supp. 3d 443 (S.D.N.Y. 2023)

Los Hechos

El abogado Steven Schwartz usó ChatGPT para investigación legal y presentó un escrito citando al menos seis decisiones judiciales inexistentes, con citas y extractos fabricados.

Ninguna de esas seis decisiones existía — el modelo las generó con total verosimilitud formal (nombres de caso, tomos, números de página).

El Fallo

El juez P. Kevin Castel encontró las fabricaciones después de que la contraparte y el propio tribunal no pudieron localizar los casos citados.

Los abogados fueron sancionados: penalidad monetaria y reprimenda pública.



El costo concreto de dejar entrar output de un LLM sin verificar a un flujo profesional de alto riesgo — nadie revisó las citas antes de radicar el escrito.

Moffatt v. Air Canada

2024 BCCRT 149 (British Columbia Civil Resolution Tribunal, feb. 2024)

Los Hechos

El chatbot del sitio web de Air Canada le dijo a un cliente que podía reservar un vuelo a tarifa completa y aplicar después un descuento por duelo de forma retroactiva.

Eso era falso: la política real exigía solicitar el descuento antes del viaje. El chatbot respondió sin contexto verificado del sistema real de políticas.

El Fallo

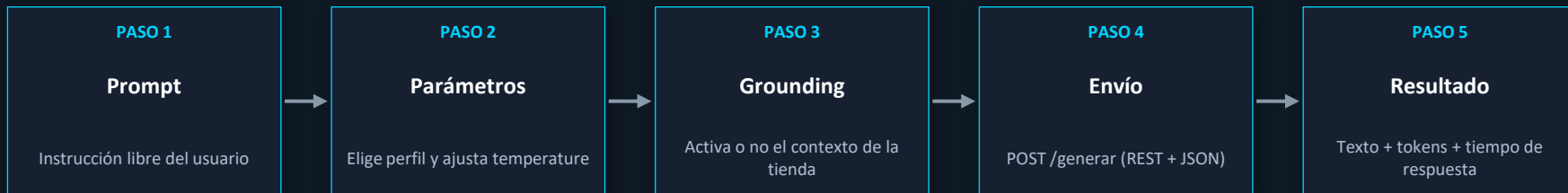
El tribunal declaró a Air Canada responsable por negligent misrepresentation (declaración negligente).

Rechazó explícitamente el argumento de la aerolínea de que el chatbot era "una entidad legal separada, responsable de sus propias acciones".

 ejemplos/03_control_alucinacion.py: la demo "Tienda Andina" de esta sesión muestra exactamente esta falla — un sistema de cara al cliente que responde sin grounding. Aquí, la consecuencia fue legal y real.

De la Teoría a la Práctica: la Demo Guiada

Tema 3 · Arquitectura del wizard de 5 pasos, un solo endpoint



Stack Técnico

Backend FastAPI (Python) — orquesta la llamada al LLM

Frontend React 18 + Vite 6 — interfaz del wizard

Comunicación REST + JSON, sin base de datos

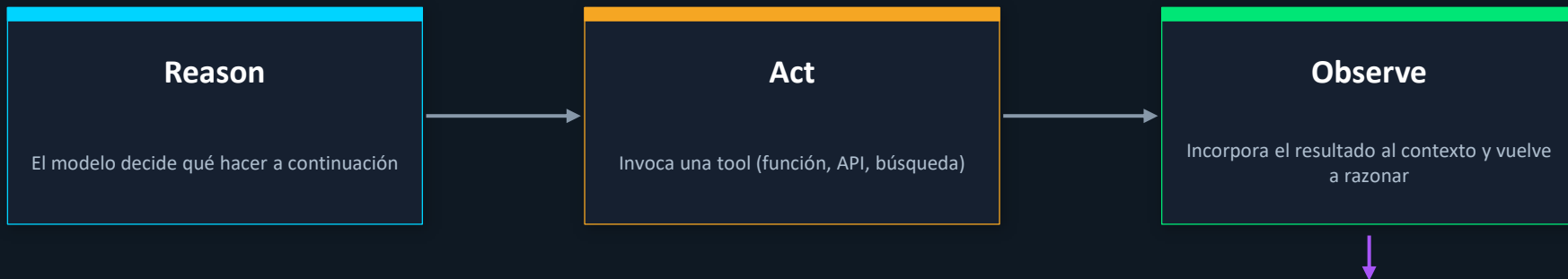
Despliegue Local para la demo — ver session_4/docs/SETUP.md

Antes de Producción (conceptual)

- ▶ Reintentos con backoff exponencial ante fallos transitorios del proveedor.
- ▶ Timeout budget explícito por request, para no bloquear el wizard.
- ▶ Circuit breaker si el proveedor externo se degrada.

Agentes como Sistemas de Decisión: el Patrón ReAct

Tema 3 · El loop de razonar, actuar y observar detrás de todo agente con tools



El ciclo Reason → Act → Observe se repite hasta resolver la tarea o alcanzar un límite de pasos.

Yao et al., 2022 — "ReAct: Synergizing Reasoning and Acting in Language Models", arXiv:2210.03629

⚠ OWASP LLM06:2025 Excessive Agency — el riesgo de gobernanza específico de los agentes: cada ciclo Act es una acción real con efectos reales. El mismo marco de seguridad de la diapositiva 8 aplica aquí, con más consecuencia.

Agentes en Google Cloud: 3 Implementaciones del Mismo Patrón

Tema 3 · Mismo loop ReAct — distinto trade-off sobre quién lo controla

Criterio	3.1 Consola	3.2 Google ADK	3.3 Docker sin ADK
Código requerido	Ninguno	Python (ADK)	Cualquiera + Docker
Quién controla el loop Reason-Act-Observe	La plataforma (GCP)	El framework (ADK)	Tú (código propio)
Control técnico	Bajo	Alto	Total
Velocidad de entrega	Minutos	Horas	Horas—días
Audiencia típica	Analistas / PM	Devs Python	Equipos con Docker
Carpeta en el repo	01_agente_consola_gcp.md	adk_agent/	sin_adk_docker/



No hay una opción "correcta": la consola valida ideas rápido, ADK da control con código, Docker da portabilidad total — la elección depende de quién debe quedarse controlando el loop.

Síntesis: un Marco Integral

El checklist que un arquitecto de LLM debería usar en cada llamada a producción

Estructura y Parámetros

- ✓ System, user, parámetros y headers separados explícitamente — nunca defaults del proveedor.
- ✓ temperature y top_p elegidos según el tipo de tarea, no copiados de un ejemplo.

Seguridad (OWASP)

- ✓ API key en variable de entorno, rate limiting activo, input sanitizado antes del prompt.

Prompting y Anti-Alucinación

- ✓ Técnica de prompting elegida por el tipo de razonamiento que la tarea exige.
- ✓ Grounding, citas o "no lo sé" activos en cualquier tarea con riesgo factual.

Gobernanza de Agentes

- ✓ Si hay tool-calling, el ciclo Reason-Act-Observe tiene límites y quién lo controla es explícito (LLM06).



Resumen de la Sesión 4



Anatomía de la llamada: system, user, parámetros, respuesta + statelessness



Seguridad bajo OWASP Top 10 para LLMs: LLM01, LLM02, LLM10 mapeados al backend



Alucinación: por qué ocurre + 5 técnicas de control + 2 casos reales con consecuencia legal



Matemática del muestreo: temperature y top_p, y su marco de decisión por tarea



Prompting con respaldo en investigación: few-shot, CoT, zero-shot-CoT, role, structured



Agentes: patrón ReAct, gobernanza OWASP LLM06, y 3 implementaciones en GCP



Recursos del Curso

Estructura de la llamada → [ejemplos/01_estructura_llamada_api.py](#)

Técnicas de prompting → [ejemplos/02_tecnicas_prompting.py](#)

Control de alucinación / Tienda Andina → [ejemplos/03_control_alucinacion.py](#)

Demo guiada → [session_4/backend/](#) · [session_4/frontend/](#) · [session_4/docs/SETUP.md](#)

Agentes en GCP → [session_4/gcp_agent_platform/](#)